

7. Implementation of thread based image processing application in Microsoft Azure

Aim: To implement thread based image processing application in Microsoft Azure

PROCEDURE:

Step 1: Create Azure Account

1. Go to Microsoft Azure
2. Sign in or create a new account
3. Open **Azure Portal**

Step 2: Create Storage Account

1. In Azure Portal, search for **Storage Accounts**
2. Click **Create**
3. Fill details:
 - Resource Group: Create new
 - Storage Account Name: imagestorage123
 - Region: Choose nearest (e.g., Central India)
4. Click **Review + Create** → **Create**

Step 3: Create Blob Container

1. Open the created Storage Account
2. Go to **Containers**
3. Click + **Container**
4. Name: images
5. Access level: Private
6. Click **Create**

Step 4: Upload Sample Images

1. Open the images container
2. Click **Upload**
3. Select multiple image files
4. Click **Upload**

Step 5: Get Connection String

1. Go to **Access Keys** in Storage Account
2. Copy **Connection String**
3. Save it for use in code

Step 6: Install Required Libraries

Run the following in your system:

```
pip install azure-storage-blob pillow
```

Step 7: Write Multithreaded Python Code

```
import threading
from azure.storage.blob import BlobServiceClient
from PIL import Image
import io

# Azure connection
connection_string = "YOUR_CONNECTION_STRING"
container_name = "images"

blob_service_client = BlobServiceClient.from_connection_string(connection_string)

def process_image(blob_name):
```

```
blob_client = blob_service_client.get_blob_client(container=container_name, blob=blob_name)

# Download image
data = blob_client.download_blob().readall()
stream = io.BytesIO(data)

# Open and process image
img = Image.open(stream)
img = img.resize((200, 200)) # Resize

# Save processed image
output = io.BytesIO()
img.save(output, format='JPEG')
output.seek(0)

# Upload processed image
new_name = "processed_" + blob_name
blob_service_client.get_blob_client(container=container_name,
blob=new_name).upload_blob(output, overwrite=True)

def main():
    container_client = blob_service_client.get_container_client(container_name)
    blobs = container_client.list_blobs()

    threads = []

    # Create threads
    for blob in blobs:
        t = threading.Thread(target=process_image, args=(blob.name,))
        threads.append(t)
        t.start()

    # Wait for completion
    for t in threads:
        t.join()

    print("All images processed successfully")

if __name__ == "__main__":
    main()
```

Step 8: Run the Application

1. Replace "YOUR_CONNECTION_STRING" with actual value
2. Run the script: **python app.py**

Step 9: Verify Output

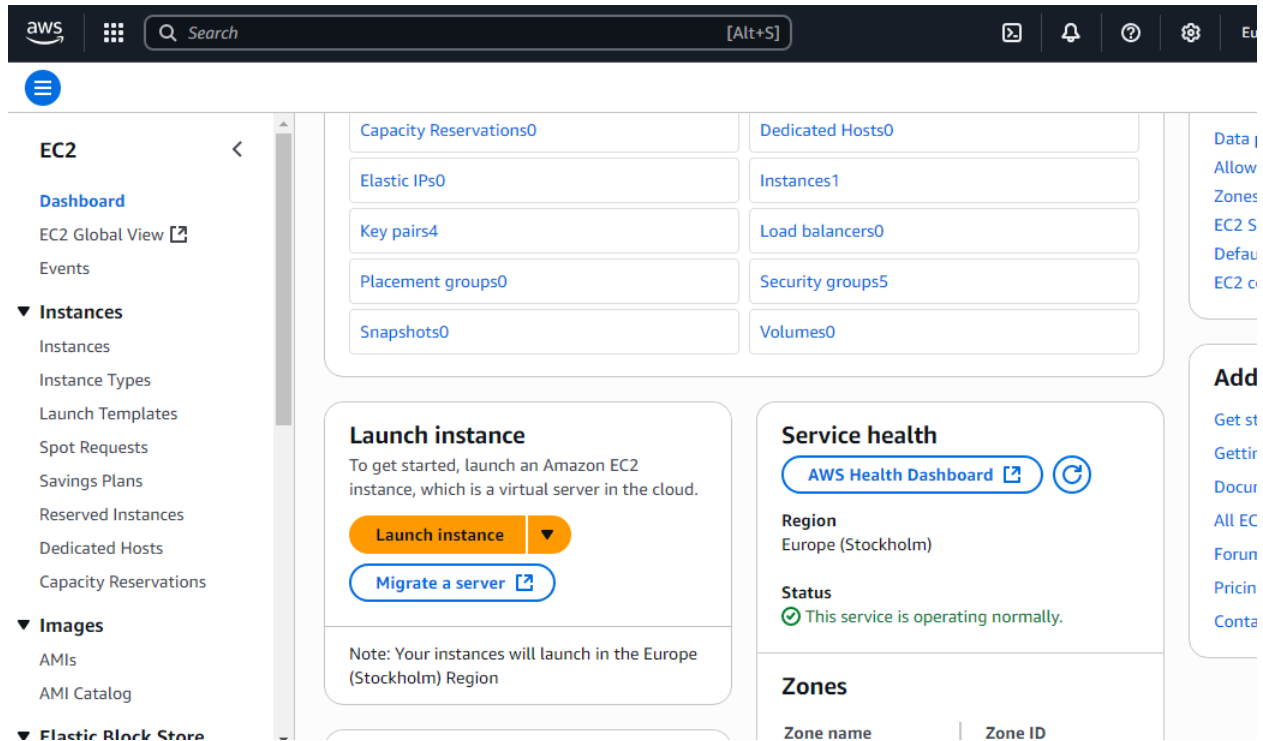
1. Go back to Azure Portal
2. Open **Blob Container (images)**
3. Check for new files:
 - processed_image1.jpg
 - processed_image2.jpg

8. Deploy dynamic web application on EC2 Instance on AWS

Aim: To deploy dynamic web application on EC2 Instance on AWS

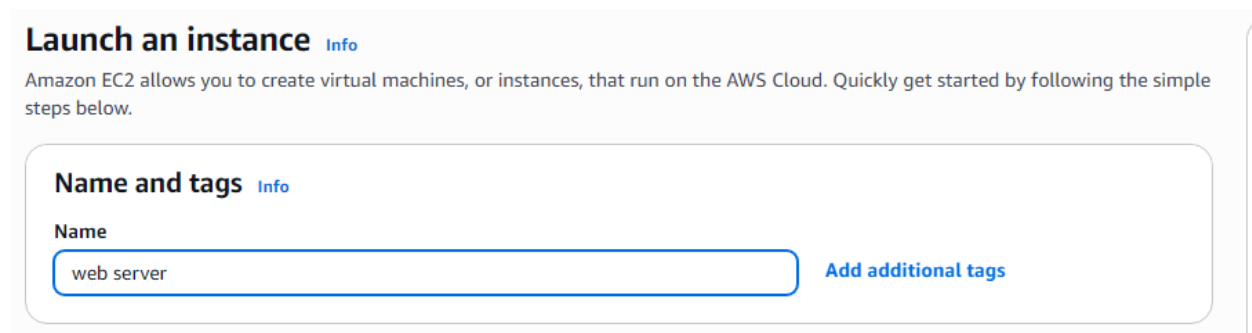
PROCEDURE:

Step 1: Launch an instance



Type EC2 in search box, Click on EC2 , Later EC2 page will open as shown above Click on Launch instance

Step 2: Name and tags



Under Name and tags give any Name like web server

Step 3: Under Application and OS images select Amazon Linux AWS

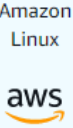
▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose **Browse more AMIs**.

Q Search our full catalog including 1000s of application and OS images

Recents

Quick Start

							 Browse more AMIs Including AMIs from AWS, Marketplace and the Community
---	---	---	---	---	--	---	--

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI
 ami-0c4fc5dcabc9df21d (64-bit (x86), uefi-preferred) / ami-07ca8bbd22d87429f (64-bit (Arm), uefi)
 Virtualization: hvm ENA enabled: true Root device type: ebs

Step 4: Under Instant type select **t3.micro**▼ Instance type [Info](#) | [Get advice](#)

Instance type

t3.micro Free tier eligible
 Family: t3 2 vCPU 1 GiB Memory Current generation: true
 On-Demand Ubuntu Pro base pricing: 0.0143 USD per Hour
 On-Demand RHEL base pricing: 0.0396 USD per Hour
 On-Demand SUSE base pricing: 0.0108 USD per Hour
 On-Demand Linux base pricing: 0.0108 USD per Hour
 On-Demand Windows base pricing: 0.02 USD per Hour

☐ All generations

[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

Step 5: Under Key Pair click on create new key Pair and give name to key Pair like kyp and click on create key pair, kyp.pm file will be downloaded.

Create key pair

Key pair name
Key pairs allow you to connect to your instance securely.

kyp

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ **RSA**
RSA encrypted private and public key pair

☐ **ED25519**
ED25519 encrypted private and public key pair

Private key file format

☒ **.pem**
For use with OpenSSH

☐ **.ppk**
For use with PuTTY

⚠ When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. [Learn](#)

[Cancel](#) [Create key pair](#)

In network settings make sure allow SSH traffic from is enabled, and enable by clicking the check box of allow HTTPS traffic from internet and check box of allow HTTP traffic from internet

▼ **Network settings** [Info](#) [Edit](#)

Network | [Info](#)
vpc-0e6c049a73374b56d

Subnet | [Info](#)
No preference (Default subnet in any availability zone)

Auto-assign public IP | [Info](#)
Enable
Additional charges apply when outside of free tier allowance

Firewall (security groups) | [Info](#)
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group ☐ Select existing security group

We'll create a new security group called 'launch-wizard-5' with the following rules:

- ☒ Allow SSH traffic from
Helps you connect to your instance Anywhere
0.0.0.0/0
- ☒ Allow HTTPS traffic from the internet
To set up an endpoint, for example when creating a web server
- ☒ Allow HTTP traffic from the internet
To set up an endpoint, for example when creating a web server

In Configure Storage keep as it is

▼ **Configure storage** [Info](#) Advanced

1x GiB ▼ Root volume, 3000 IOPS, Not encrypted

Free tier eligible customers can get up to 30 GB of EBS General Purpose (SSD) or Magnetic storage

[Add new volume](#)

In Advanced details keep as it is.

EC2 > [Instances](#) > Launch an instance

▼ **Configure storage** [Info](#) Advanced

1x GiB ▼ Root volume, 3000 IOPS, Not encrypted

Free tier eligible customers can get up to 30 GB of EBS General Purpose (SSD) or Magnetic storage

[Add new volume](#)

Click refresh to view backup information
The tags that you assign determine whether the instance will be backed up by any Data Lifecycle Manager policies.

0 x File systems
[Edit](#)

► **Advanced details** [Info](#)

▼ **Summary**

Number of instances [Info](#)
1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.8.2...[read more](#)
ami-0c4fc5dcabc9df21d

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

Again click on Launch Instance

aws [Alt+S]

EC2

Dashboard
EC2 Global View [↗](#)
Events

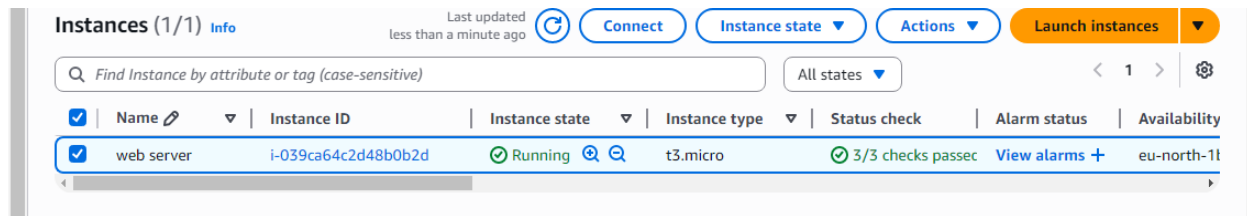
▼ **Instances**
Instances
Instance Types
Launch Templates

Resources [EC2 Global View](#) [⚙️](#) [🔄](#)

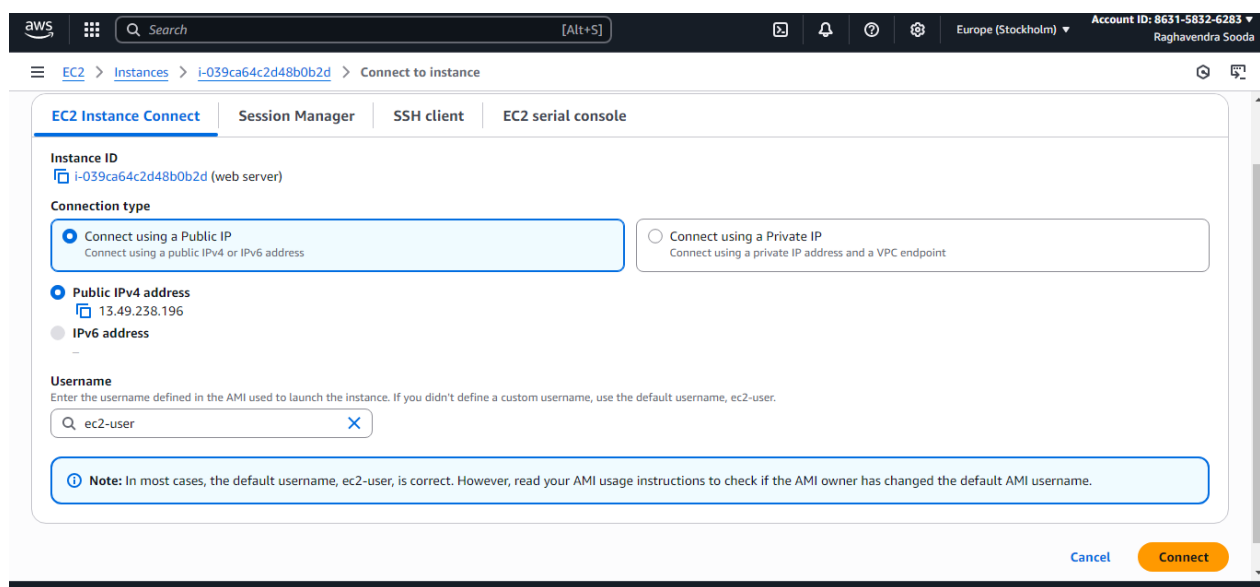
You are using the following Amazon EC2 resources in the Europe (Stockholm) Region:

Instances (running)0	Auto Scaling Groups0
Capacity Reservations0	Dedicated Hosts0
Elastic IPs0	Instances1
Key pairs4	Load balancers0

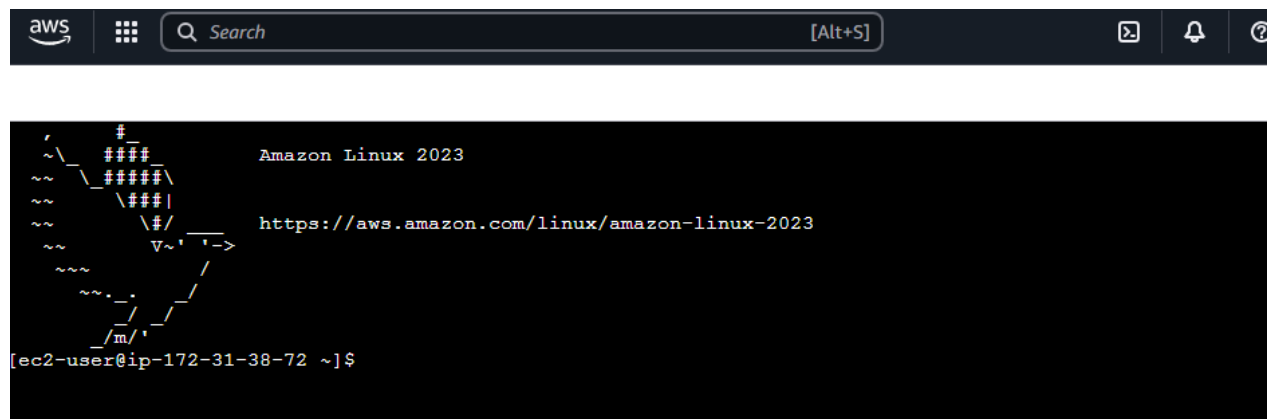
After Successful Launching click on Instances



Select the check box of newly selected instance named web- server click on connect button



Under Connect select EC2 Instance Connect method and then press connect button.



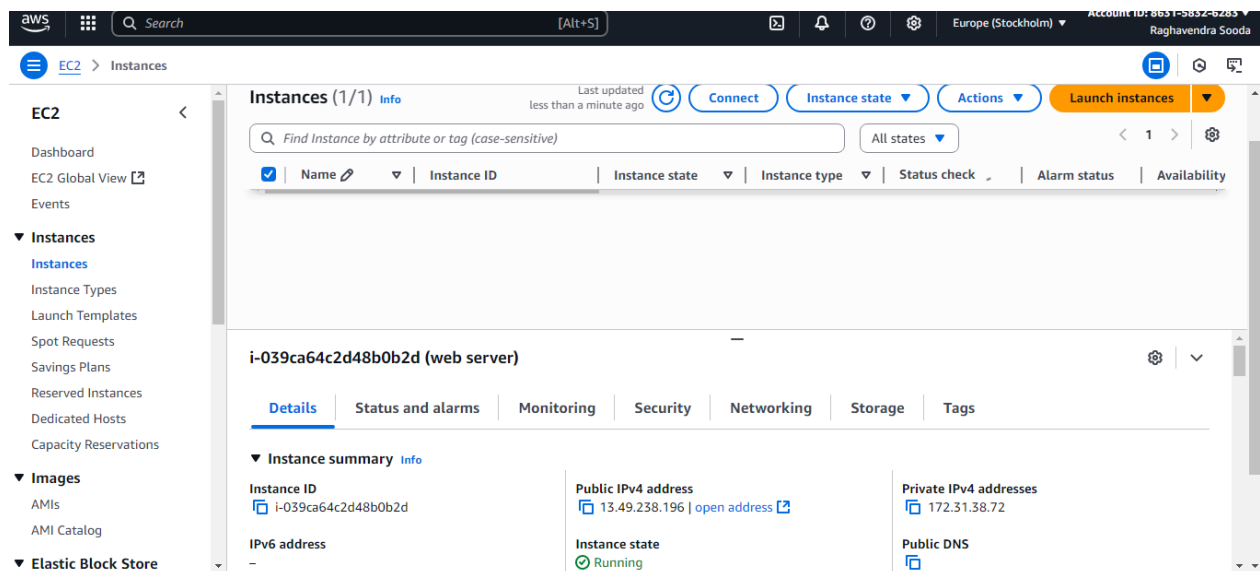
Type following steps

```

sudo su - ;root user
yum update -y :update
yum install -y httpd ; install httpd server
mkdir temp
cd temp
wget https://templatemo.com/download/templatemo_596_electric_xtra ;download website
template files
ls -lrt
mkdir templatemo_596_electric_xtra_unzipped
unzip templatemo_596_electric_xtra -d templatemo_596_electric_xtra_unzipped
cd templatemo_596_electric_xtra_unzipped
ls -lrt
cd templatemo_596_electric_xtra
ls -lrt
mv * /var/www/html/ copy the extracte file
/var/www/html cd /var/www/html/
ls -lrt
systemctl status httpd ; to check the status of httpd
systemctl enable httpd ;enable the httpd server
systemctl start httpd ; start the httpd server

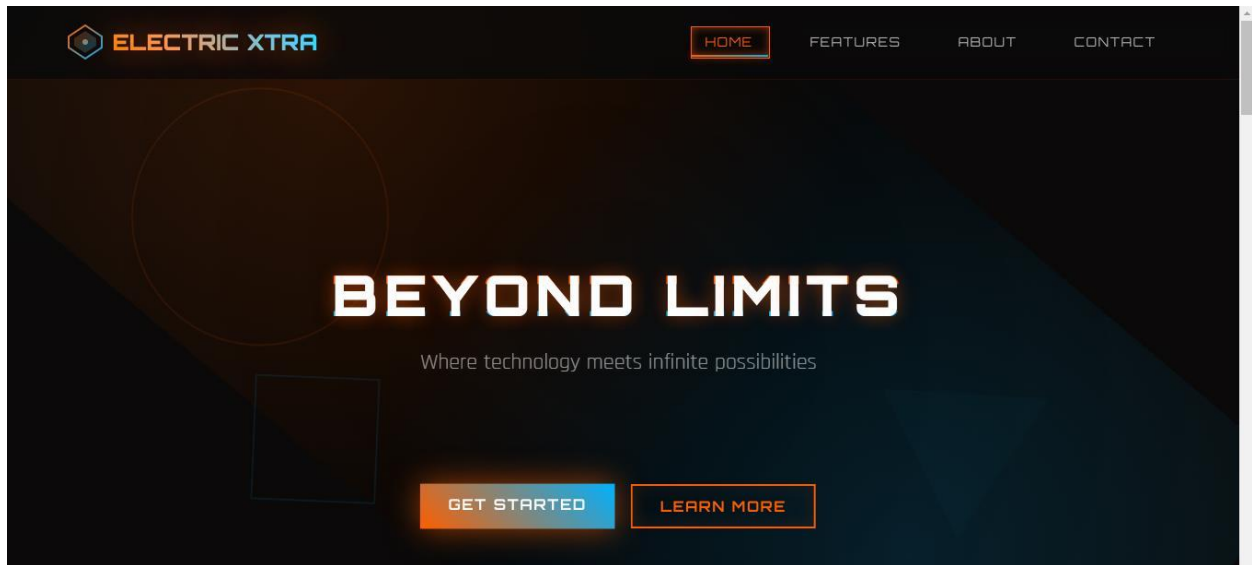
```

click on the instances



copy the public ipv4 address 13.49.238.1969 (in my case) go to browser window paste

the copied public ip of the instance. Web page will open as shown below



9. Use Google App Engine Launcher to launch web applications.

Aim: To use Google App Engine Launcher to launch web applications.

PROCEDURE:

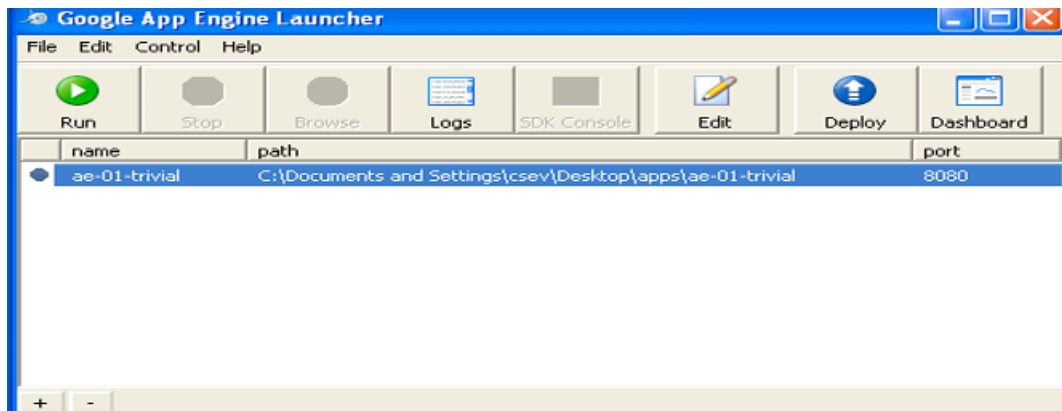
C:\Documents and Settings\csev\Desktop\apps And then make a sub---folder in within apps called “ae--01--trivial” – the path to this folder would be: C:\ Documents and Settings \csev\Desktop\apps\ae--01--trivial Using a text editor such as JEdit (www.jedit.org)

create a file called app.yaml in the ae--01--trivial folder with the following contents: application: ae-01-trivial version: 1 runtime: python api_version: 1 handlers:- url: /* script: index.py

Then create a file in the ae--01--trivial folder called index.py with three lines in it: print 'Content-Type: text/plain' print '' print 'Hello there Chuck'

Then start the GoogleAppEngineLauncher program that can be found under Applications. Use the File --> Add Existing Application command and navigate into the apps directory and select the ae-01--trivial folder.

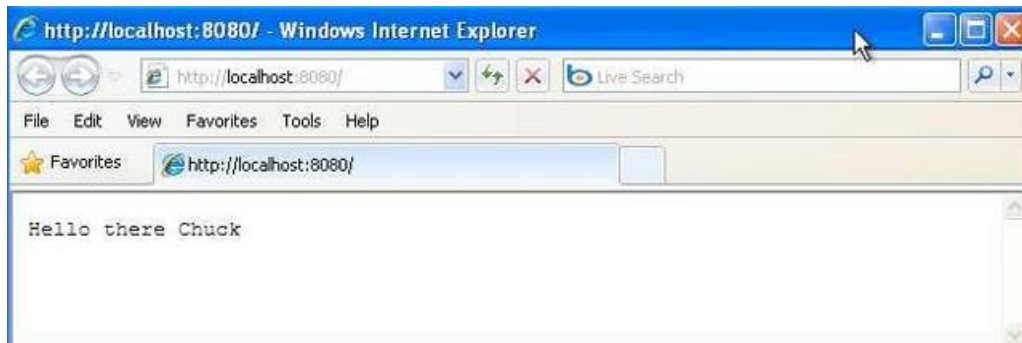
Once you have added the application, select it so that you can control the application using the launcher.



Once you have selected your application and press Run. After a few moments your application will start and the launcher will show a little green icon next to your application.

Then press Browse to open a browser pointing at your application which is running at <http://localhost:8080/>

Paste <http://localhost:8080> into your browser and you should see your application as follows:



Edit the index.py to change the name “Chuck” to your own name and press Refresh in the browser to verify your updates.

Watching the Log You can watch the internal log of the actions that the web server is performing when you are interacting with your application in the browser.

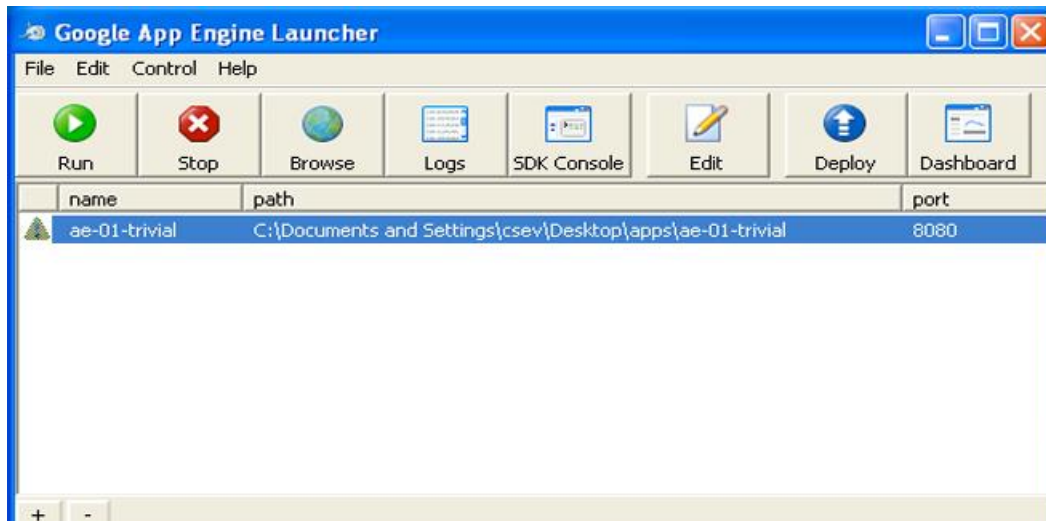
Select your application in the Launcher and press the Logs button to bring up a log window:

Each time you press Refresh in your browser – you can see it retrieving the output with a GET request.



Dealing with Errors

With two files to edit, there are two general categories of errors that you may encounter. If you make a mistake on the app.yaml file, the App Engine will not start, and your launcher will show a yellow icon near your application:



To get more detail on what is going wrong, take a look at the log for the application:



Result:

Thus the GAE web applications were created.

10. Deploy static web application using S3 on AWS and secure it with signed URL's

Aim: To deploy static web application using S3 on AWS and secure it with signed URL's

PROCEDURE:

Step 1: Create AWS S3 bucket:

- On AWS console move to S3 and create bucket,
- Select AWS region
- Enter the Bucket name
- Uncheck the "Block all public access"
- leave the rest of the settings default and press Create Bucket

The screenshot shows the 'Create bucket' page in the AWS Management Console. The breadcrumb trail at the top is 'Amazon S3 > Buckets > Create bucket'. The main heading is 'Create bucket' with an 'Info' link. Below this, a note states 'Buckets are containers for data stored in S3.' with a 'Learn more' link. The 'General configuration' section contains a dropdown for 'AWS Region' set to 'US East (Ohio) us-east-2', a text input for 'Bucket name' containing 'deploy-app-on-s3', and a 'Choose bucket' button. A note below the bucket name states it must be unique and follow naming rules, with a link to 'See rules for bucket naming'. The 'Object Ownership' section has two radio buttons: 'ACLs disabled (recommended)' (selected) and 'ACLs enabled'. The 'ACLs disabled' option explains that all objects are owned by the account and access is specified using only policies. The 'ACLs enabled' option explains that objects can be owned by other AWS accounts and access can be specified using ACLs. At the bottom, it shows 'Object Ownership' is set to 'Bucket owner enforced'.

Step 2: Once it is created then select "Properties", scroll down to bottom, "Edit" the static website hosting and set the below settings and press "Save changes",

Amazon S3 > Buckets > [deploy-app-on-s3](#) > Edit static website hosting

Edit static website hosting [Info](#)

Static website hosting
Use this bucket to host a website or redirect requests. [Learn more](#)

Static website hosting

☐ Disable

☒ Enable

Hosting type

☒ Host a static website
Use the bucket endpoint as the web address. [Learn more](#)

☐ Redirect requests for an object
Redirect requests to another bucket or domain. [Learn more](#)

For your customers to access content at the website endpoint, you must make all your content publicly readable. To do so, you can edit the S3 Block Public Access settings for the bucket. For more information, see [Using Amazon S3 Block Public Access](#)

Index document
Specify the home or default page of the website.

Error document - *optional*
This is returned when an error occurs.

Step 3: Then, select "Permissions", then "Edit" the Bucket policy, add the following content (remember to update the BUCKET_NAME) and Save Changes,

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::BUCKET_NAME/*",
```

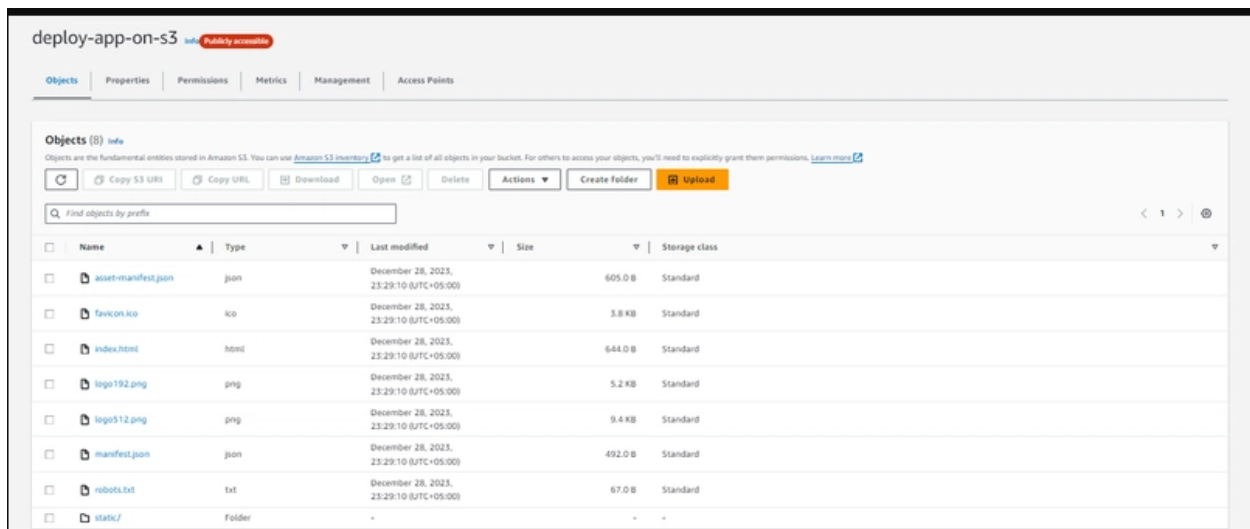
```

    "arn:aws:s3:::BUCKET_NAME"
  ]
}
]
}

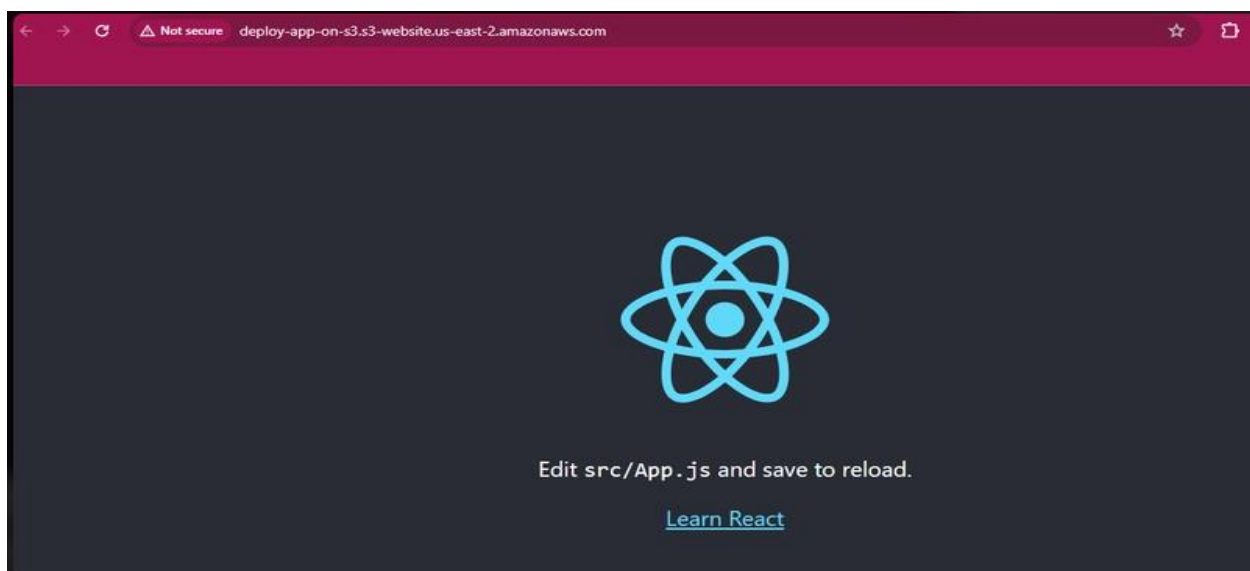
```

Step 4: Upload Your Files:

- Open your new bucket and click Upload.
- Drag and drop your website files (e.g., index.html, styles.css) and click Upload.



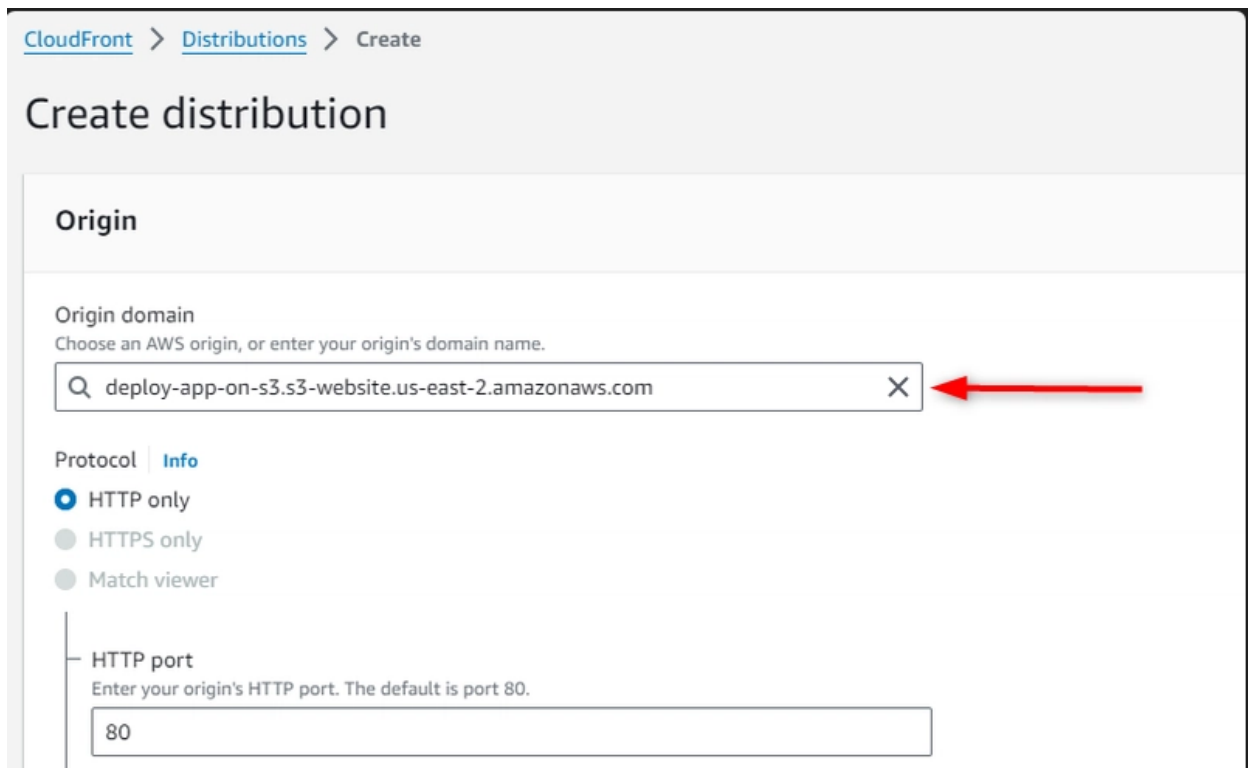
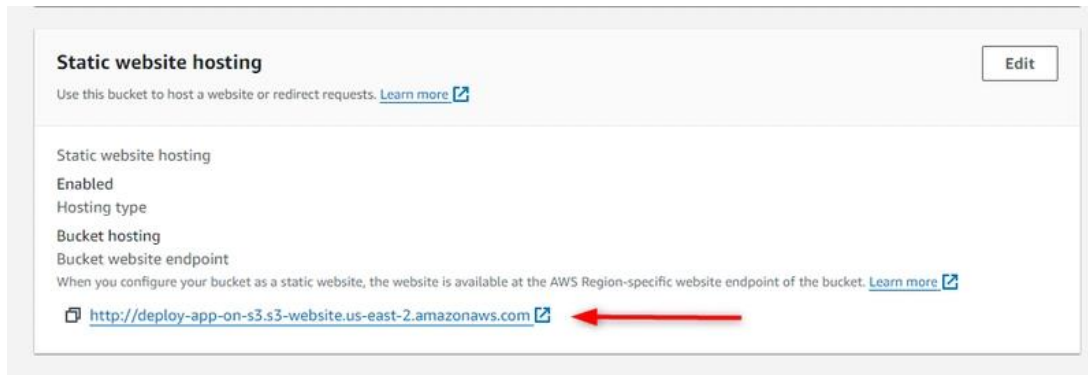
Now the static app will be accessible on the S3 static url,



Step 5: Attach Cloudfront with S3:

To access the static content from S3 with high speed and less latency we will attach the cloudfront with S3 bucket. To do so,

- move to Cloudfront from AWS console. Press the "Create Distribution"
- Copy the static url from the s3 bucket properties and place it in the origin domain for cloudfront. Don't forget to remove the https:// from the s3 bucket static url. Image description



- Then, select the Redirect HTTP to HTTPS and set the Allowed HTTP methods:

Viewer

Viewer protocol policy

☐ HTTP and HTTPS

☒ Redirect HTTP to HTTPS

☐ HTTPS only

Allowed HTTP methods

☐ GET, HEAD

☐ GET, HEAD, OPTIONS

☒ GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE

Cache HTTP methods

GET and HEAD methods are cached by default.

☒ OPTIONS

Restrict viewer access

If you restrict viewer access, viewers must use CloudFront signed URLs or signed cookies to access your content.

☒ No

☐ Yes

- Then press "Create Distribution"
- Once the distribution is created, you will be able to access your website on Distribution domain name:

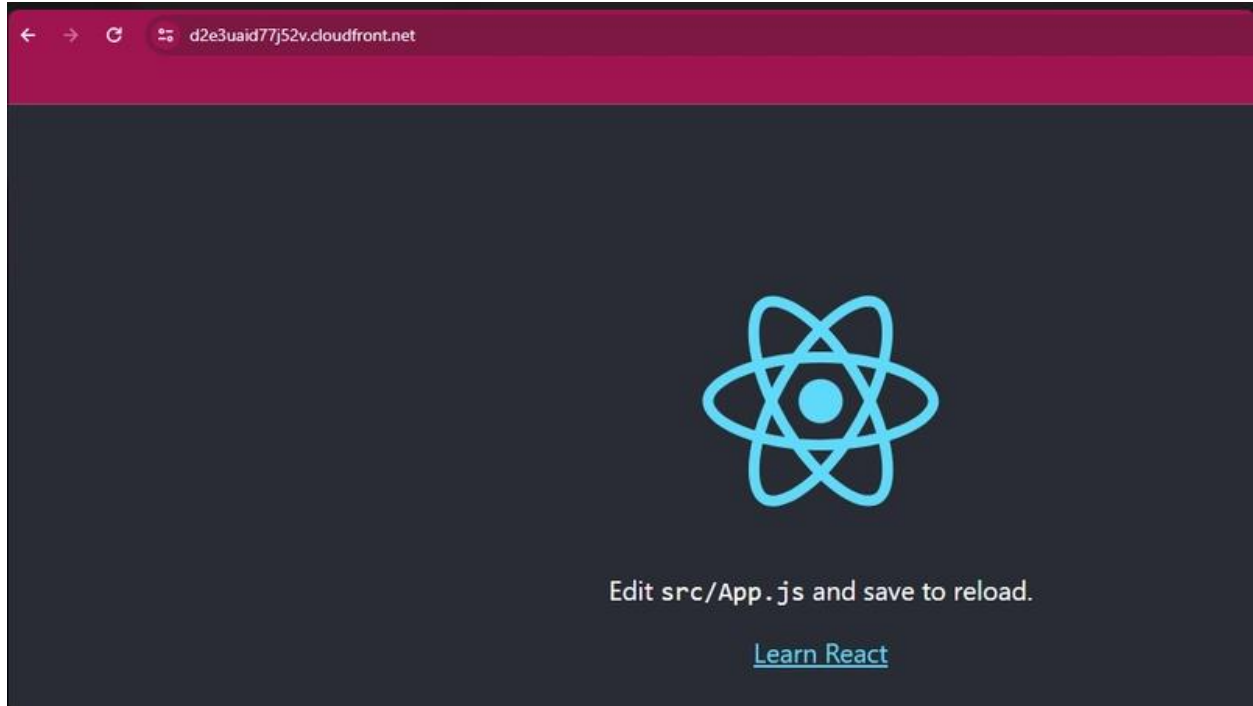
CloudFront > Distributions > E29HVFXACX6YMV

E29HVFXACX6YMV

General | Security | Origins | Behaviors | Error pages | Invalidations | Tags

Details

Distribution domain name	ARN
 d2e3uaid77j52v.cloudfront.net	 arn:aws:cloudfront::260923197968:distribution/E29HVFXACX6YMV



If you created your static website only as a learning exercise, delete the AWS resources that you allocated so that you no longer accrue charges. After you delete your AWS resources, your website is no longer available.

11. Create Video streaming service using S3 and cloudfront and AWS Elemental Media Convert Digital Rights Management System

Step 1: Set Up Your Amazon S3 Bucket

Storing raw video assets in a secure S3 bucket prevents unauthorized direct downloads while remaining available for your CDN.

1. Login to AWS Management Console. Navigate to: Services → S3
2. Click **Create bucket**.
 - Enter a globally unique name (e.g., my-video-streaming-bucket-2026).
 - Select your preferred AWS Region (e.g., us-east-1 or your closest localized region).
3. **Configure Settings**:
 - **Block Public Access**: Ensure **Block all public access** remains checked. Your bucket must remain private to force users through CloudFront.
 - **Bucket Versioning**: Select **Enable**.
 - **Default Encryption**: Select **Enable** with SSE-S3.
4. Click the **Create bucket** button at the bottom.
5. **Upload a Video**: Click on your new bucket name, select **Upload**, add a test .mp4 video, and finish by clicking **Upload**.

Step 2: Create a CloudFront Distribution

CloudFront acts as the CDN, which retrieves your private S3 objects securely and distributes them to your users.

1. Search for and open the **CloudFront** console.
2. Click **Create distribution**.
3. **Origin Settings**:
 - **Origin domain**: Select the S3 bucket you created in Step 1.
 - **Origin access**: Select **Origin access control settings (recommended)**.
 - **Origin access control (OAC)**: Click **Create control setting**, leave the default settings, and hit **Create**. *(This securely connects CloudFront to your private S3 bucket).*
4. **Default Cache Behavior Settings**:
 - **Viewer protocol policy**: Select **Redirect HTTP to HTTPS**.
 - **Allowed HTTP methods**: Select GET, HEAD.
 - **Cache key and origin requests**: Choose **Cache policy and origin request policy (recommended)** -> Cache policy: CachingOptimized.
5. **Web Application Firewall (Optional)**: Select **Do not enable security protections** (for basic lab purposes).
6. **Create Distribution**: Click **Create distribution** at the bottom of the page.

Step 3: Update S3 Bucket Policy for CloudFront

Because your S3 bucket is private, you need to grant CloudFront permission to access the video files.

1. Once your CloudFront distribution is created, a banner will appear at the top prompting you to **Copy policy**. Copy the policy JSON provided.
2. Go back to your **S3 console** and click on your bucket.
3. Select the **Permissions** tab.
4. Scroll down to **Bucket policy** and click **Edit**.
5. Paste the copied CloudFront policy into the editor and click **Save changes**.

Example:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipal",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::video-streaming-bucket/*"
    }
  ]
}
```

Step 4: Test Your Streaming Service

1. Go back to your **CloudFront distributions** page and wait for the status to change from *Deploying* to *Last modified* (with a green checkmark).
2. Copy the **Distribution domain name** (e.g., d111111abcdef8.cloudfront.net).
3. Open a new web browser tab, paste the domain name, and append the name of the uploaded video file to the end of the URL (e.g., https://cloudfront.net).
4. The video should load immediately in the browser via the CloudFront stream.